Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation

Press, Smith and Lewis (ICLR 2022), arXiv:2108.12409

Paper summary for the CS-781 capstone

The paper at a glance

Citation

O. Press, N. A. Smith, M. Lewis. *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*. ICLR 2022. University of Washington, Facebook AI Research, Allen Institute for AI.

One-sentence summary

Drop position embeddings entirely. Subtract a fixed, head-specific penalty proportional to query–key **distance** from each attention score. Models trained on short inputs then work on longer ones.

Why it matters for our capstone

Our A3 prefix-LM uses ALiBi and has no position embeddings. Legal judgments are long, so extrapolation matters for Phases 4–7.

Where it appears in our slides

Main deck: “A3: hybrid prefix-LM with ALiBi”. Glossary: “Positional information (3)”.

Roadmap

1. The problem: extrapolation
2. The basic idea: linear biases
3. Results
4. Assessment and outlook

Can a model read longer inputs than it was trained on?

Extrapolation: performing well when the input at inference is longer than the training length L.

Why we want it:

- Attention costs $O(L^2)$, so training on long sequences is expensive.
- If a model could train on $L = 512$ and still read 2,048 tokens later, training would be much cheaper.

Vaswani et al. speculated that sinusoidal positions "may" extrapolate. **Nobody had checked carefully.**

Legal motivation

Our Phase-1 models train with a 1,024-token window. A typical Supreme Court judgment is many thousands of tokens. For judgment prediction or case retrieval, we would like to feed more of the document at inference without retraining on long sequences.

Existing position methods, tested for extrapolation

Setup: WikiText-103 language model (Baevski and Auli, 2018: 16 layers, $d = 1024$, 8 heads). Train with $L = 512$ or 1024; evaluate on longer inputs, changing *only* the position method.

- **Learned positions**: no embedding exists beyond L, so extrapolation is impossible.
- **Sinusoidal**: improves for up to about 20–50 extra tokens, then degrades.
- **Rotary (RoPE)**: better; gains up to about 100–200 extra tokens. It was slower in their implementation.
- **T5 bias**: best; gains up to about 600–800 extra tokens, but training is at least **twice as slow**.

Key finding

Extrapolation depends heavily on the **position method**.

Methods that inject position *at every layer into attention*, and not into the values, extrapolate better. ALiBi builds on this observation.

No efficiency gain from T5 bias

T5 bias at $L = 512$ runs at 14.4k words/s; sinusoidal at $L = 1024$ runs at 28.5k. Training short with T5 bias saves nothing.

ALiBi: attention with linear biases

No position embeddings anywhere. For query i attending to keys $1 \dots i$, add a fixed bias:

$$\text{softmax}\big(q_i K^\top + m \cdot [-(i-1), \dots, -2, -1, 0]\big)$$

- Penalty grows **linearly** with distance.
- m is a **head-specific slope**, fixed, not learned.
- Position enters every layer, through attention scores only; values carry no position.

Implementation: add the bias to the attention mask, a few lines of code. There is no runtime penalty; the only extra is a per-head mask ($n \times L \times L$, up to about 100 MB).

Bias matrix for one head, $m = 1$:

	k_1	k_2	k_3	k_4	k_5
q_1	0				
q_2	-1	0			
q_3	-2	-1	0		
q_4	-3	-2	-1	0	
q_5	-4	-3	-2	-1	0

Multiply by the head's slope m and add to $q_i \cdot k_j$.
Blank cells are masked (future).

Choosing the slopes

For n heads, the slopes form a geometric sequence that starts at $2^{-8/n}$ with ratio $2^{-8/n}$.

- 8 heads: $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots, \frac{1}{256}$.
- 16 heads: $\frac{1}{2^{0.5}}, \frac{1}{2}, \frac{1}{2^{1.5}}, \ldots, \frac{1}{2^{8}}$.

Findings from about ten slope sets tried:

- The best sets have slopes in $(0, 1)$, denser near 0.
- The method is robust to the exact choice; the same slopes worked across domains and sizes.
- **Trainable slopes** extrapolated poorly and slowed training by 3%.

Different heads, different ranges

- Slope $\frac{1}{2}$: a key 10 tokens back loses 5 points. A **local** head, for grammar.
- Slope $\frac{1}{256}$: 10 tokens back loses only 0.04. A **long-range** head, for topic, parties and sections.

Our A3 (12 heads)

12 is not a power of two. As the ALiBi code does, we take the 8-head slopes plus every other 16-head slope: 0.707, 0.354, 0.177 and 0.088.

Easy worked example

Query at position 5; keys at positions 1–5 with raw scores $q \cdot k = [2.0,\ 1.0,\ 1.5,\ 0.5,\ 1.0]$. Distances $[4, 3, 2, 1, 0]$.

Key position	1	2	3	4	5
Raw weight (no bias)	**0.39**	0.14	0.24	0.09	0.14
Bias, $m = 0.5$	-2	-1.5	-1	-0.5	0
Biased score	0.0	-0.5	0.5	0.0	1.0
ALiBi weight	0.14	0.09	0.24	0.14	**0.39**

What happened

With slope 0.5, the farthest key drops from 0.39 to 0.14, and the nearest key rises from 0.14 to 0.39. A strongly matching distant key can still win, but it must match *much* better.

With slope $1/256$ the weights barely change: that head still looks far back.

Why it extrapolates

A distance of 1,500 needs no new parameters: the penalty is just $1500\,m$. Very distant keys are simply faded out.

WikiText-103: better, even without extrapolating

- For every training length L, ALiBi beats sinusoidal *at the same length*. For example, at $L = 3072$: **17.60** vs 18.67 perplexity.
- ALiBi trained on $L = 512$ scores 19.73 at 512 tokens, and **18.40** when evaluated on 3,072 tokens. That beats the sinusoidal model *trained* on 3,072.
- That $L = 512$ model trains **1.84**$\times$ **faster** and fits on a smaller GPU.
- At $L = 1024$ and 3072, ALiBi also beats rotary and T5 bias without extrapolation.
- The $L = 512$ model keeps improving up to inputs of over 12k tokens.

Books domain

The same slopes, unchanged, were used on the Toronto Book Corpus. Results transferred fully: better than sinusoidal both with and without extrapolation.

Recency bias

ALiBi prefers nearby tokens. For language modelling on small data this inductive bias helps even at equal length.

Scaling up: 1.3B parameters on 461 GB

Model: 25 layers, 16 heads, $d = 2048$, FFN 8192: **1.3B parameters**. Data: RoBERTa corpus + English CC-100 (461 GB). One epoch = 50k updates on 128 V100 GPUs.

- ALiBi trained on $L = 1024$ **beats** sinusoidal trained on $L = 2048$ at 2,048 tokens by 0.09 perplexity, using 3.1 GB less memory.
- It reaches a given perplexity about **11% faster**.
- ALiBi $L = 512$ vs sinusoidal $L = 1024$ at 1,024 tokens: only 0.06 behind, 7% faster, 1.6 GB less memory.
- At equal L, ALiBi and sinusoidal are *similar* at this scale.

The "$2L$" sweet spot

On this corpus, the best perplexity comes at about **twice** the training length:

- $L = 512$: 9.79 at 512, best 9.3 at 1,012 tokens.
- $L = 1024$: 9.16 at 1,024, best 8.9 at 2,024 tokens.

Performance stays strong up to 10,000 tokens. Sinusoidal degrades as soon as one token is added beyond L.

Why does longer input help? The early-token curse

Evaluation usually splits text into non-overlapping chunks of length L_{valid}. Tokens at the *start* of each chunk have little context, so they get worse predictions: the **early-token curse**.

Two possible reasons ALiBi improves with longer inputs:

1. it really uses context more than L tokens back; or
2. longer chunks mean fewer tokens suffer the early-token curse.

Using sliding-window evaluation, the authors find that ALiBi's gain is **largely explained by reason 2**.

An honest caveat

ALiBi lets a model *accept* long inputs gracefully. It does not prove the model *uses* information from thousands of tokens back. The authors suggest future work should exploit longer histories more efficiently.

For our evaluation

Our continuation loss always gives the model 768 tokens of context, so it avoids the early-token curse by design.

ALiBi in our capstone

- A3 uses ALiBi with 12 heads and no position embeddings.
- **Our extension**: the paper uses causal attention only. A3 also has a *bidirectional* prefix, so we use the symmetric penalty $-m\,|i-j|$ inside the prompt.
- The bias is built as a float mask together with the prefix mask. Our tests check it against cached generation.

Experiments worth running

- Evaluate A1 (learned), A3 (ALiBi) and A4 (RoPE) at 1,024, 1,536 and 2,048 tokens. A1 cannot go beyond 1,024 at all.
- Train A3 at $L = 512$ and test at 1,024: can it match a 1,024-trained model, as in the paper?
- For long judgments in Phases 4–7, compare ALiBi extrapolation against chunking.

Strengths and limitations

Strengths

- Extremely simple; no parameters; no runtime cost.
- Real extrapolation: train short, test long.
- Saves training time and memory at a given test length.
- Better perplexity than sinusoidal on smaller data.
- Slopes transfer across domains and model sizes.
- Careful, controlled comparison of four position methods.

Limitations

- Best results come at about $2L$; much longer inputs help less.
- Gains largely come from avoiding the early-token curse, not from long-range use.
- The recency bias may hurt tasks that need precise distant retrieval.
- Designed and tested for causal language modelling only.
- At billion-parameter scale, no gain at equal length.
- Slopes are hand-chosen.

Future research directions

From the paper

- Exploit longer histories more efficiently, beyond reducing the early-token curse.
- Combine ALiBi with orthogonal methods such as kNN-LM or Routing Transformer.
- Use the saved memory to stack more layers.

What followed

- Adopted in large open models such as BLOOM (BigScience, 2022) and MPT.
- Length-extrapolation research: RoPE scaling methods (Position Interpolation, YaRN), xPos, and studies of positional schemes such as NoPE (Kazemnejad et al., 2023).

Open questions

- Can a model *use* information thousands of tokens back, not just tolerate it?
- Should slopes adapt to the task or the domain?
- How does recency bias interact with retrieval of a single far-away fact ("needle in a haystack")?
- For legal text: parties and statutes named on page 1 matter on page 40. Is a linear recency penalty the right prior for judgments?

Summary

- Sinusoidal positions do not extrapolate; rotary helps a little; T5 bias helps more but is slow.
- **ALiBi**: no position embeddings; add $-m \cdot$distance to each attention score, with fixed geometric slopes per head.
- Worked example: slope 0.5 moves attention from the farthest key (0.39) to the nearest (0.39).
- Results: better perplexity at equal length on WikiText-103; a model trained on 512 tokens beats a sinusoidal model trained on 3,072; at 1.3B, train on 1,024 and match or beat 2,048, 11% faster.
- Best around $2L$; gains mostly from reducing the early-token curse.
- For us: the position scheme of A3, extended to a bidirectional prefix.

Reference

Press, O., Smith, N. A., and Lewis, M. (2022). Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *International Conference on Learning Representations (ICLR)*. arXiv:2108.12409.

Other work cited: Vaswani et al. (2017); Raffel et al. (2020) T5; Su et al. (2021) RoFormer; Press et al. (2021) Shortformer; BigScience (2022) BLOOM; Chen et al. (2023) Position Interpolation; Peng et al. (2023) YaRN; Sun et al. (2023) xPos; Kazemnejad et al. (2023) The Impact of Positional Encoding on Length Generalization in Transformers.